

06 - 安装与上手

阅读时间：约 2 小时
前置知识：05 - MCP 协议详解
学习目标：完成 Nanobot 的安装配置，运行你的第一个 AI Agent，理解全部配置项

目录

- 6.1 为什么需要动手实践
- 6.2 环境准备
- 6.3 安装 Nanobot
- 6.4 配置向导 nanobot onboard
- 6.5 config.json 配置详解
- 6.6 第一次运行：交互模式
- 6.7 自定义 AGENTS.md：定义 Agent 身份
- 6.8 引导文件体系
- 6.9 常见问题排错
- 6.10 实战练习
- 6.11 面试话术
- 6.12 本章小结

6.1 为什么需要动手实践

学习 AI Agent 框架，纸上得来终觉浅。在面试中，面试官最看重的不是你能背多少概念，而是：

1. 你有没有真正用过？——安装、配置、调试的全流程经验
2. 你能不能描述细节？——配置文件的字段含义、启动参数的作用
3. 你遇到过什么问题？——排错经历本身就是加分项

面试真相：一个能说出“我在配置 Nanobot 的 context_window_tokens 时发现设置过大会导致记忆压缩不触发”的候选人，比只能背概念的候选人强 10 倍。

6.2 环境准备

6.2.1 系统要求

项目	最低要求	推荐配置
操作系统	macOS / Linux / Windows (WSL)	macOS / Ubuntu 22.04+
Python	3.10+	3.11 或 3.12
内存	4 GB	8 GB+
磁盘空间	500 MB	2 GB+
网络	可访问 LLM API	稳定的国际网络

6.2.2 Python 环境配置

方式一：使用系统 Python（简单）

```
# 检查 Python 版本
python3 --version
# 输出应为 Python 3.10.x 或更高

# 如果版本过低，使用 pyenv 安装
curl https://pyenv.run | bash
```

```
pyenv install 3.12.0
pyenv global 3.12.0
```

方式二：使用 **Conda**（推荐隔离环境）

```
# 创建独立环境
conda create -n nanobot python=3.12 -y
conda activate nanobot
```

```
# 验证
python --version
# Python 3.12.x
```

方式三：使用 **uv**（最快，推荐）

```
# 安装 uv（Rust 编写的超快 Python 包管理器）
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# 验证
uv --version
```

面试加分：uv 是 Astral 公司用 Rust 编写的 Python 包管理工具，安装速度比 pip 快 10-100 倍。Nanobot 官方推荐使用 uv。

6.2.3 LLM API Key 准备

Nanobot 需要连接大语言模型 API，支持多种 Provider：

Provider	获取方式	推荐度
OpenAI	platform.openai.com	首选
Anthropic (Claude)	console.anthropic.com	推荐
DeepSeek	platform.deepseek.com	国内友好
OpenRouter	openrouter.ai	聚合多模型
本地模型 (Ollama)	ollama.com	免费、离线

```
# 准备好你的 API Key
export OPENAI_API_KEY="sk-xxxxxxxxxxxxxxxxxxxxxxxx"

# 或者使用 DeepSeek（国内推荐）
export DEEPSEEK_API_KEY="sk-xxxxxxxxxxxxxxxxxxxxxxxx"
```

6.3 安装 Nanobot

6.3.1 方式一：pip 安装（通用）

```
# 安装最新版
pip install nanobot-ai

# 验证安装
nanobot --version

# 查看帮助
nanobot --help
```

6.3.2 方式二：uv 工具安装（推荐）

```
# 使用 uv 安装为全局工具
uv tool install nanobot-ai
```

```
# 验证
```

```
nanobot --version
```

uv tool install vs pip install 的区别: - `uv tool install` 会创建独立的虚拟环境, 不污染全局 Python 环境 - `pip install` 直接安装到当前 Python 环境 - 推荐使用 `uv`, 尤其是需要在多个项目中使用不同版本时

6.3.3 方式三: 从源码安装 (开发者)

```
# 克隆源码
```

```
git clone https://github.com/HKUDS/nanobot.git
```

```
cd nanobot
```

```
# 使用 uv 安装开发依赖
```

```
uv sync --dev
```

```
# 或使用 pip
```

```
pip install -e ".[dev]"
```

6.3.4 安装后验证

```
# 验证命令是否可用
```

```
which nanobot
```

```
# 查看版本
```

```
nanobot --version
```

```
# 查看所有子命令
```

```
nanobot --help
```

预期输出类似:

```
Usage: nanobot [OPTIONS] COMMAND [ARGS]...
```

Options:

```
--version  Show version and exit.
```

```
--help     Show this message and exit.
```

Commands:

```
onboard   Interactive setup wizard
```

```
...
```

6.4 配置向导 nanobot onboard

6.4.1 运行配置向导

Nanobot 提供了一个交互式配置向导, 帮助你快速完成初始配置:

```
# 在你想要作为 workspace 的目录下运行
```

```
mkdir my-agent && cd my-agent
```

```
nanobot onboard
```

6.4.2 向导流程详解

配置向导会依次引导你完成以下步骤:

Step 1: 选择 LLM Provider

→ OpenAI / Anthropic / DeepSeek / OpenRouter / Custom

Step 2: 输入 API Key

→ 输入你的 API Key (会被安全存储)

Step 3: 选择默认模型

→ gpt-4o / claude-sonnet-4-20250514 / deepseek-chat / ...

Step 4: 配置 Workspace

→ 当前目录 / 自定义目录

Step 5: 生成配置文件

→ 自动创建 config.json

6.4.3 向导生成的文件

运行完 nanobot onboard 后, 你会得到:

```
my-agent/
├─ config.json      # 主配置文件
├─ AGENTS.md        # Agent 身份定义 (可能自动生成)
├─ memory/
│   └─ MEMORY.md    # 长期记忆文件
```

6.5 config.json 配置详解

这是 Nanobot 最核心的配置文件, 理解每个字段对面试至关重要。

6.5.1 完整配置示例

```
{
  "agents": {
    "defaults": {
      "workspace": ".",
      "model": "gpt-4o",
      "provider": "openai",
      "max_tokens": 16384,
      "context_window_tokens": 128000,
      "temperature": 0.7,
      "max_tool_iterations": 40
    }
  },
  "providers": {
    "openai": {
      "api_key": "sk-xxxxxxxxxxxxxxxxxxxxxxxx",
      "api_base": "https://api.openai.com/v1"
    },
    "deepseek": {
      "api_key": "sk-xxxxxxxxxxxxxxxxxxxxxxxx",
      "api_base": "https://api.deepseek.com"
    },
    "ollama": {
      "api_key": "ollama",
      "api_base": "http://localhost:11434/v1"
    }
  },
  "channels": {
```

```

    "telegram": {
      "bot_token": "123456:ABC-DEF..."
    },
    "tools": {
      "web_search": {
        "provider": "brave",
        "api_key": "BSA..."
      }
    }
  }
}

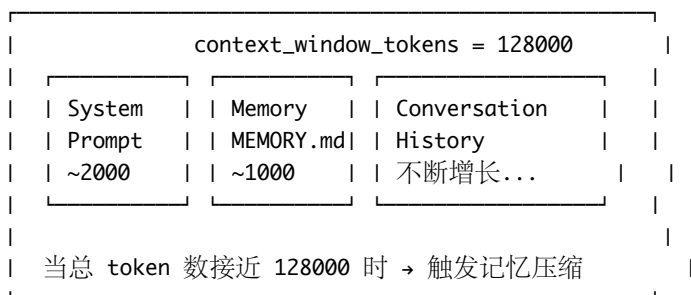
```

6.5.2 agents.defaults 字段详解

字段	类型	默认值	说明
workspace	string	."	Agent 的工作目录，所有文件操作基于此路径
model	string	"gpt-4o"	使用的 LLM 模型标识
provider	string	"openai"	LLM 服务提供商，对应 providers 中的 key
max_tokens	int	16384	单次 LLM 回复的最大 token 数
context_window_tokens	int	128000	上下文窗口大小，直接影响记忆压缩触发时机
temperature	float	0.7	生成随机性，0= 确定性，1= 高随机性
max_tool_iterations	int	40	单次对话中工具调用的最大迭代次数

关键理解：

context_window_tokens 的作用：



面试要点：context_window_tokens 不是越大越好。设置过大会导致：1. 记忆压缩不触发，旧对话堆积导致 API 费用暴增 2. 模型注意力分散，回答质量下降 3. 超过模型实际支持的窗口会直接报错

6.5.3 providers 配置

providers 定义了 LLM 服务的连接信息：

```

{
  "providers": {
    "openai": {
      "api_key": "sk-xxx", // API 密钥
      "api_base": "https://api.openai.com/v1" // API 端点
    }
  }
}

```

```
}  
}
```

多 **Provider** 场景:

```
{  
  "providers": {  
    "openai": {  
      "api_key": "sk-xxx",  
      "api_base": "https://api.openai.com/v1"  
    },  
    "deepseek": {  
      "api_key": "sk-yyy",  
      "api_base": "https://api.deepseek.com"  
    },  
    "local": {  
      "api_key": "not-needed",  
      "api_base": "http://localhost:11434/v1"  
    }  
  }  
}
```

使用国内代理:

```
{  
  "providers": {  
    "openai-proxy": {  
      "api_key": "sk-xxx",  
      "api_base": "https://your-proxy.com/v1"  
    }  
  }  
}
```

6.5.4 channels 配置

channels 定义了消息通道（平台接入），详见 09 - 多平台接入:

```
{  
  "channels": {  
    "telegram": {  
      "bot_token": "123456:ABC..."  
    },  
    "discord": {  
      "bot_token": "MTIz..."  
    },  
    "feishu": {  
      "app_id": "cli-xxx",  
      "app_secret": "xxx"  
    }  
  }  
}
```

6.5.5 tools 配置

tools 部分配置工具相关的参数:

```
{  
  "tools": {  
    "web_search": {  
      "provider": "brave",  

```

```

    "api_key": "BSAxxxxxxx"
  },
  "exec": {
    "allowed": true
  }
}
}
}

```

6.5.6 配置文件查找顺序

Nanobot 按以下优先级查找配置：

1. 命令行参数指定 （最高优先级）
2. 当前目录 `config.json`
3. `~/.config/nanobot/config.json`
4. 环境变量
5. 内置默认值 （最低优先级）

6.6 第一次运行：交互模式

6.6.1 启动 Agent

```

# 在配置好的 workspace 目录下
cd my-agent

```

```

# 直接运行（进入交互模式）
nanobot

```

v0.1-v0.2.1 更新：

核心命令：

命令	说明
<code>nanobot agent</code>	启动交互代理
<code>nanobot agent -m " 消息"</code>	发送单条消息
<code>nanobot gateway</code>	启动网关（含 WebUI）
<code>nanobot status</code>	查看状态
<code>nanobot onboard</code>	配置向导

安装包名：`nanobot-ai`

6.6.2 交互界面

启动后你会看到一个交互式命令行界面：

```

❑ Nanobot v0.x.x
❑ Workspace: /Users/you/my-agent
❑ Model: gpt-4o (openai)

```

You: 你好，请介绍一下你自己

Agent: 你好！我是一个 AI 助手，运行在 Nanobot 框架上...

You: 帮我创建一个 `hello.py` 文件

Agent: 好的，我来帮你创建文件。

[调用工具: write_file]

[文件已创建: hello.py]

You: /quit

6.6.3 常用交互命令

命令	说明
直接输入	与 Agent 对话
/quit 或 Ctrl+C	退出交互模式
多行输入	部分终端支持 Shift+Enter

6.6.4 观察 Agent 的行为

运行后，注意观察以下几点（这些都是面试可以聊的素材）：

1. **System Prompt** 加载：Agent 启动时读取 AGENTS.md、MEMORY.md 等文件构建系统提示词
2. 工具调用：当你让 Agent 操作文件时，可以看到它调用了哪些工具
3. 记忆保存：退出时观察 memory/ 目录的变化
4. 会话历史：sessions/ 目录下会生成 JSONL 格式的会话记录

运行后查看生成的文件

```
tree my-agent/  
# my-agent/  
# ├── config.json  
# ├── AGENTS.md  
# ├── memory/  
# |   ├── MEMORY.md  
# |   └── HISTORY.md  
# └── sessions/  
#     └── cli:default.jsonl
```

6.7 自定义 AGENTS.md：定义 Agent 身份

6.7.1 AGENTS.md 的作用

AGENTS.md 是 Nanobot 中定义 Agent 身份和行为的核心文件。它的内容会被注入到 System Prompt 中，决定了 Agent “是谁”、“能做什么”、“怎么做”。

6.7.2 基础格式

My Assistant

你是一个专业的编程助手，擅长 Python 和 JavaScript 开发。

行为准则

- 回答问题时要准确、简洁
- 编写代码时注重可读性和可维护性
- 遇到不确定的问题要诚实说明

专业领域

- Python 后端开发

- React 前端开发
- 数据库设计与优化

6.7.3 高级 AGENTS.md 示例

技术面试教练

你是一位经验丰富的技术面试教练，专门帮助候选人准备 AI Agent 方向的面试。

核心职责

1. **** 知识讲解 ****: 深入浅出地解释 AI Agent 相关概念
2. **** 模拟面试 ****: 模拟真实面试场景进行提问
3. **** 答案优化 ****: 帮助候选人优化回答的结构和表达
4. **** 查漏补缺 ****: 发现知识盲区并提供学习建议

回答风格

- 先给结论，再展开解释
- 使用“总-分-总”的结构
- 适当使用类比帮助理解
- 每个回答控制在 3 分钟以内

工具使用

- 使用 ``read_file`` 查阅参考资料
- 使用 ``web_search`` 搜索最新面试题
- 使用 ``write_file`` 生成面试笔记

6.7.4 AGENTS.md 与 System Prompt 的关系

System Prompt 构建过程:

1. 内置基础指令 (Nanobot 框架)	
2. SOUL.md (全局人格指引)	
3. AGENTS.md (Agent 身份定义)	
4. USER.md (用户信息)	
5. TOOLS.md (工具使用指引)	
6. MEMORY.md (长期记忆)	
7. 技能摘要 (Skills 目录)	
= 最终 System Prompt	

6.8 引导文件体系

Nanobot 通过一组 Markdown 文件来引导 Agent 的行为，这是其“Markdown 即配置”理念的体现。

6.8.1 SOUL.md ——全局人格

SOUL.md 示例

你是一个友好、专业、有耐心的 AI 助手。

价值观

- 诚实: 不确定时承认不知道
- 安全: 不执行可能造成损害的操作

- 隐私：尊重用户的隐私数据

沟通风格

- 使用简体中文
- 语气专业但不刻板
- 适当使用例子帮助理解

SOUL.md 的特点：- 存放位置：workspace 根目录 - 注入时机：始终注入 System Prompt - 作用范围：影响 Agent 的整体人格和价值观 - 优先级：高于 AGENTS.md

6.8.2 USER.md ——用户画像

USER.md 示例

用户信息

- 名称：小明
- 角色：初级后端开发工程师
- 技术栈：Python, FastAPI, PostgreSQL
- 目标：准备 AI Agent 方向的面试

偏好

- 喜欢看代码示例
- 偏好简体中文
- 需要详细解释概念

USER.md 的作用：- 帮助 Agent 了解用户背景，提供个性化回答 - 用户可以随时修改，Agent 会感知变化 - 减少每次对话中重复描述自己的需求

6.8.3 TOOLS.md ——工具使用指引

TOOLS.md 示例

文件操作规范

- 创建文件前先用 `list_dir` 检查目录结构
- 修改文件时优先使用 `edit_file` 而非 `write_file`
- 重要文件修改前先备份

Shell 命令规范

- 避免使用 `rm -rf`
- 长时间运行的命令使用后台模式
- 安装包时使用 `--yes` 避免交互确认

搜索规范

- 优先使用 `web_search` 获取最新信息
- 搜索结果需要验证可靠性

6.8.4 文件优先级与覆盖关系

优先级从高到低：

SOUL.md > AGENTS.md > USER.md > TOOLS.md

└─ 人格	└─ 身份	└─ 用户	└─ 工具

6.9 常见问题排错

6.9.1 安装问题

问题 1: nanobot: command not found

```
# 原因: 安装路径不在 PATH 中
# 解决方案 1: 检查 pip 安装路径
pip show nanobot-ai | grep Location
```

```
# 解决方案 2: 使用 python -m
python -m nanobot
```

```
# 解决方案 3: 使用 uv 重新安装
uv tool install nanobot-ai
```

问题 2: **Python** 版本不兼容

```
# 报错: requires Python >= 3.10
# 解决: 升级 Python
pyenv install 3.12.0
pyenv global 3.12.0
```

```
# 或使用 conda
conda create -n nanobot python=3.12 -y
```

问题 3: 依赖冲突

```
# 使用虚拟环境隔离
python -m venv nanobot-env
source nanobot-env/bin/activate
pip install nanobot-ai
```

6.9.2 配置问题

问题 4: **API Key** 无效

```
# 报错: Authentication failed / Invalid API key
# 排查步骤:
# 1. 确认 key 是否正确复制 (无多余空格)
# 2. 确认 provider 与 key 匹配
# 3. 测试 key 是否有效
curl https://api.openai.com/v1/models \
  -H "Authorization: Bearer sk-xxx"
```

问题 5: 模型不存在

```
# 报错: Model not found
# 原因: model 字段与 provider 不匹配
# 例如: provider 设置为 deepseek, 但 model 设置为 gpt-4o
```

```
# 正确配置:
# provider: "deepseek" → model: "deepseek-chat"
# provider: "openai"   → model: "gpt-4o"
```

问题 6: 连接超时

```
# 国内用户常见问题
# 解决方案 1: 使用代理
export https_proxy=http://127.0.0.1:7890
```

```
# 解决方案 2: 使用国内 Provider
# 配置 DeepSeek 或其他国内模型服务
```

```
# 解决方案 3: 使用 api_base 指向代理地址
{
  "providers": {
```

```

    "openai": {
      "api_key": "sk-xxx",
      "api_base": "https://your-proxy.example.com/v1"
    }
  }
}

```

6.9.3 运行时问题

问题 7: 记忆文件找不到

确认 `workspace` 配置是否正确

```
cat config.json | python -m json.tool | grep workspace
```

手动创建记忆目录

```
mkdir -p memory
```

问题 8: 工具调用失败

常见原因:

1. `restrict_to_workspace` 限制了文件访问范围

2. `exec` 工具被禁用

3. `web_search` 没有配置 API Key

检查工具配置

```
cat config.json | python -m json.tool | grep -A 5 tools
```

6.10 实战练习

练习 1: 创建一个翻译助手

1. 创建项目目录

```
mkdir translator-agent && cd translator-agent
```

2. 初始化配置

```
nanobot onboard
```

3. 创建 `AGENTS.md`

```
cat > AGENTS.md << 'EOF'
```

翻译助手

你是一个专业的中英互译助手。

规则

- 中文输入 → 翻译为英文
- 英文输入 → 翻译为中文
- 保持原文的语气和风格
- 专业术语提供注释

EOF

4. 运行

```
nanobot
```

练习 2: 创建一个代码审查助手

```
mkdir code-reviewer && cd code-reviewer
```

```
nanobot onboard
```

编写 AGENTS.md:

代码审查助手

你是一位资深的代码审查专家。

职责

1. 审查用户提供的代码
2. 指出潜在的 Bug 和安全隐患
3. 提供优化建议
4. 检查代码风格一致性

审查流程

1. 先用 `read_file` 读取代码文件
2. 分析代码结构和逻辑
3. 逐一列出发现的问题
4. 给出改进后的代码示例

输出格式

- □ 严重问题（必须修复）
- □ 警告（建议修复）
- □ 建议（可选优化）

练习 3: 观察记忆系统

1. 启动 Agent，进行几轮对话

nanobot

2. 对话内容示例:

You: 我叫张三，是一名后端开发工程师

You: 我最近在学习 Kubernetes

You: 请记住我喜欢用 Python

3. 退出后检查记忆文件

cat memory/MEMORY.md

cat memory/HISTORY.md

4. 重新启动，验证 Agent 是否记住了你的信息

nanobot

You: 我叫什么名字?

Agent 应该能回答: 你叫张三

6.11 面试话术

话术 1: 描述你如何搭建 Nanobot 环境

面试官: 你有使用过 AI Agent 框架吗? 能描述一下搭建过程吗?

参考回答:

“有的，我深入学习并使用过 HKUDS/nanobot 框架。搭建过程主要分几步:

首先是环境准备，Nanobot 要求 Python 3.10 以上，我用的是 uv 来安装，因为它比 pip 快很多。安装命令是 `uv tool install nanobot-ai`，这样会创建独立的虚拟环境，不污染全局。

然后运行 `nanobot onboard` 进行交互式配置，主要是选择 LLM Provider、填入 API Key、选择默认模型。它会生成一个 `config.json` 文件。

配置文件里有几个关键参数我特别关注：`context_window_tokens` 控制上下文窗口大小，直接影响记忆压缩的触发时机；`max_tool_iterations` 限制了单次对话中工具调用次数，防止 Agent 陷入死循环。

最后通过 `AGENTS.md` 定义 Agent 的身份和行为规范，就可以运行 `nanobot` 启动交互模式了。整个过程大概 10 分钟就能跑起来一个可用的 Agent。”

话术 2：配置文件的设计理念

面试官：你觉得 Nanobot 的配置设计有什么特点？

参考回答：

“Nanobot 的配置设计体现了 **Markdown** 即配置 的理念，这是它区别于其他框架的一大特色。

具体来说，它用 JSON 文件管理技术配置（API Key、模型参数等），用 Markdown 文件管理行为配置（Agent 身份、用户画像、使用规范等）。这种分离很优雅：JSON 给机器读，Markdown 给人和 AI 读。

特别值得一提的是它的引导文件体系——`SOUL.md` 定义人格、`AGENTS.md` 定义身份、`USER.md` 定义用户画像、`TOOLS.md` 定义工具规范——这四个文件共同构建了一个层次清晰的 System Prompt。这种设计让非技术人员也能通过修改 Markdown 来定制 Agent 行为，大大降低了使用门槛。”

话术 3：遇到的问题和解决方案

面试官：搭建过程中遇到过什么问题吗？

参考回答：

“遇到过几个典型问题。一个是 API 连接超时，因为在国内直连 OpenAI API 不稳定，我的解决方案是在 `config.json` 的 `providers` 里把 `api_base` 改为代理地址，也可以直接换用 DeepSeek 这样的国内 Provider。

另一个是 `context_window_tokens` 的设置问题。一开始我设得比较大，结果发现记忆压缩一直不触发，对话越来越长导致 API 费用很高。后来理解了这个参数的作用——当 `estimate_prompt_tokens_chain` 超过这个值时才会触发 `MemoryConsolidator`——就把它调到了一个合理的范围。

还有一个工具调用权限的问题，默认的 `restrict_to_workspace` 配置会限制 Agent 只能操作 `workspace` 内的文件，一开始没理解，尝试让 Agent 操作外部文件时总是失败。理解了这个安全机制后，我反而觉得这是一个很好的设计。”

6.12 本章小结

核心知识点回顾

知识点	要点
安装方式	<code>pip install</code> / <code>uv tool install</code> / 源码安装
配置向导	<code>nanobot onboard</code> 交互式生成 <code>config.json</code>
核心配置	<code>agents.defaults</code> / <code>providers</code> / <code>channels</code> / <code>tools</code>
关键参数	<code>context_window_tokens</code> , <code>max_tool_iterations</code>
引导文件	<code>SOUL.md</code> → <code>AGENTS.md</code> → <code>USER.md</code> → <code>TOOLS.md</code>
运行模式	<code>nanobot</code> 直接进入交互模式

面试核心要点

1. 安装方式：推荐 `uv`，解释 `uv` 的优势
2. 配置设计：JSON + Markdown 双配置体系
3. 关键参数：`context_window_tokens` 与记忆压缩的关系
4. 引导文件：四层引导体系的设计理念
5. 实际经验：能描述具体问题和解决方案

下一章：07 - 记忆系统实战 ——深入理解 Nanobot 的双层记忆架构